Top/Arange 60/60

SMART GRID TECHNOLOGIES
COUSEWORK

FEBRUARY 16, 2017

YUAN LI
01228509

Yuan Li - 01228509

Part A

1.

To compute the availabilities of the five components T1, T2, F1, F2 and F3, we can use the relation $availability = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF+MTTR}$. This results in:

$$a_{T1} = a_{T2} = \frac{100 * 365}{100 * 365 + 10} = \frac{3650}{3651}$$

$$2{\times}0.05 = 0.1year^{-1} = \frac{1}{MTTF_{F1}}$$

$$MTTF_{F1} = 10$$

$$a_{F1} = \frac{10 * 365}{10 * 365 + 2} = \frac{1825}{1826}$$

$$3{\times}0.05 = 0.15year^{-1} = \frac{1}{MTTF_{F2}}$$

$$a_{F2} = \frac{\frac{20}{3} * 365}{\frac{20}{3} * 365 + 2} = \frac{3650}{3653}$$

$$6{\times}0.1 = 0.6year^{-1} = \frac{1}{MTTF_{F3}}$$

$$a_{F3} = \frac{\frac{5}{3} * 8760}{\frac{5}{3} * 8760 + 10} = \frac{1460}{1461}$$

2.

To compute the probability that all load is being supplied, we must distinguish between situation where a single transformer can supply all the load (load<=8WM) and situation where both transformers are required to supply the load(load>8WM). Note that the power factor is unit throughout. Therefore, 8MVA is equal to 8MW. The reliability block diagrams are shown below:

Summary of Comments on Microsoft Word - Smart_Grid_Coursework_Yuan_Li_01228509.docx

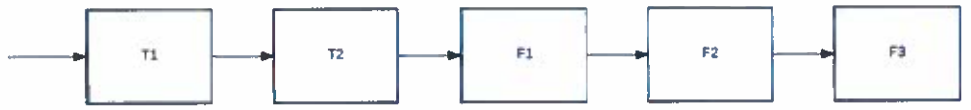
Page: 2

Number: 1 Author: Simon Tindemans Date: Indeterminate

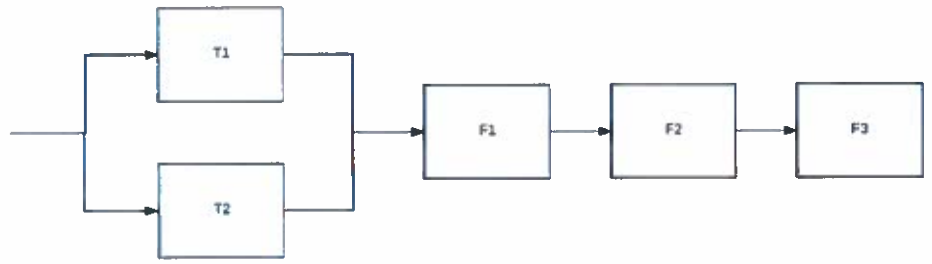
Simon Tindemans: answers are correct, but don't forget to mention units consistently.

Yuan Li - 01228509

Total load is larger than 8WM



Total load is smaller than 8WM



$\Pr(all\ load\ supplied)$
$= \Pr(load \leq 8WM)\ \Pr(F1\ is\ available)\ \Pr(F2\ is\ available)\ \Pr(F3\ is\ available)\ \Pr(T1\ or\ T2\ is\ available)$
$+ \Pr(load > 8WM)\ \Pr(F1\ is\ available)\ \Pr(F2\ is\ available)\ \Pr(F3\ is\ available)\ \Pr(T1\ is\ available)$
$\Pr(T2\ is\ available)$

$$= \frac{20}{24} * \left(1 - \left(1 - \frac{3650}{3651}\right) * \left(1 - \frac{3650}{3651}\right)\right) * \frac{1825}{1826} * \frac{3650}{3653} * \frac{1460}{1461} + \frac{4}{24}$$
$$* \left(\frac{3650}{3651} * \frac{3650}{3651}\right) * \frac{1825}{1826} * \frac{3650}{3653} * \frac{1460}{1461} = 0.99786$$
$$LOLP = 1 - 0.99786 = 0.00214$$

ok

3.

The fact that transformers will trip when overloaded. This means two situations dependent on load level requires to be considered as well. It is worth mention that there are two situations when load is larger than 8WM because load will become smaller than 8WM when other feeders are unavailable.

$\Pr(point\ a\ being\ supplied)$
$= \Pr(load \leq 8WM)\ \Pr(F1\ is\ available)\ \Pr(T1\ or\ T2\ is\ available)$
$+ \Pr(load > 8WM)\ [\Pr(F1\ is\ available)\ \Pr(T1\ and\ T2\ is\ available)$
$+ \Pr(F1\ is\ available)\ \Pr(F2\ is\ unavailable\ or\ F3\ is\ unavailable)\ \Pr(only\ one\ transfomer\ is\ available)]$

$$= \frac{20}{24} * \left(1 - \left(1 - \frac{3650}{3651}\right) * \left(1 - \frac{3650}{3651}\right)\right) * \frac{1825}{1826} + \frac{4}{24}$$
$$* \left(\frac{3650}{3651} * \frac{3650}{3651} * \frac{1825}{1826} + \left(\frac{3650}{3651} * \frac{1}{3651} + \frac{3650}{3651} * \frac{1}{3651}\right) * \frac{1825}{1826} * \left(1 - \frac{3650}{3653} * \frac{1460}{1461}\right)\right)$$
$$= 0.99936$$

$$\Pr(point\ a\ not\ supplied) = 1 - 0.99936 = 0.00064$$

Yuan Li - 01228509

$$\Pr(\textit{point b being supplied})$$
$$= \Pr(\textit{load} \le 8WM)\ \Pr(F2\ \textit{is available})\ \Pr(T1\ \textit{or}\ T2\ \textit{is available})$$
$$+ \Pr(\textit{load} > 8WM)\ [\Pr(F2\ \textit{is available})\ \Pr(T1\ \textit{and}\ T2\ \textit{is available})$$
$$+ \Pr(F2\ \textit{is available})\ \Pr(F1\ \textit{is unavailable or}\ F3\ \textit{is unavailable})\ \Pr(\textit{only one transfomer is available})]$$
$$= \frac{20}{24} * \left(1 - \left(1 - \frac{3650}{3651}\right) * \left(1 - \frac{3650}{3651}\right)\right) * \frac{3650}{3653} + \frac{4}{24}$$
$$* \left(\frac{3650}{3651} * \frac{3650}{3651} * \frac{3650}{3653} + \left(\frac{3650}{3651} * \frac{1}{3651} + \frac{3650}{3651} * \frac{1}{3651}\right) * \frac{3650}{3653} * \left(1 - \frac{1825}{1826} * \frac{1460}{1461}\right)\right.$$
$$= 0.99908$$

$$\Pr(\textit{point a not supplied}) = 1 - 0.99908 = 0.00092$$

$$\Pr(\textit{point b being supplied})$$
$$= \Pr(\textit{load} \le 8WM)\ \Pr(F2\ \textit{is available})\ \Pr(F3\ \textit{is available})\ \Pr(T1\ \textit{or}\ T2\ \textit{is available})$$
$$+ \Pr(\textit{load} > 8WM)\ [\Pr(F2\ \textit{is available})\ \Pr(F3\ \textit{is available})\ \Pr(T1\ \textit{and}\ T2\ \textit{is available})$$
$$+ \Pr(F2\ \textit{is available})\ \Pr(F3\ \textit{is available})\ \Pr(F1\ \textit{is unavailable})\ \Pr(\textit{only one transfomer is available})]$$
$$= \frac{20}{24} * \left(1 - \left(1 - \frac{3650}{3651}\right) * \left(1 - \frac{3650}{3651}\right)\right) * \frac{3650}{3653} * \frac{1460}{1461} + \frac{4}{24}$$
$$* \left(\frac{3650}{3651} * \frac{3650}{3651} * \frac{3650}{3653} * \frac{1460}{1461} + \left(\frac{3650}{3651} * \frac{1}{3651} + \frac{3650}{3651} * \frac{1}{3651}\right) * \frac{3650}{3653} * \frac{1460}{1461} * (1\right.$$
$$\left. - \frac{1825}{1826}\right) = 0.99840$$

$$\Pr(\textit{point a not supplied}) = 1 - 0.9984 = 0.0016$$

ok

4.

- The CML is computed as

$$\text{CML} = \frac{\sum_{loadpoints} N_{customers\ at\ loadpoint} \times \text{unavailability}_{\text{loadpoint}}}{N_{customers}} \times 60 min/h \times 365 day$$
$$= 60 \times 365 \times 24 \times \left\{\frac{5000[1 - \Pr(a)] + 4000[1 - \Pr(b)] + 2000[1 - \Pr(c)]}{11000}\right\}$$
$$= 481.64 \text{minutes}$$

ok

- Analytical EENS Computation[1]

The ~~analytical value of EENS can be~~ calculated by the following equation:

$$EENS = (\Pr(\textit{load} \le 8WM) * \Pr(\textit{point a is unavailable underloaded}))$$
$$* \text{average load per household per hour} + \Pr(\textit{load} \ge 8WM)$$
$$* \Pr(\textit{point a is unavailable overloaded})$$
$$* \text{average load per household per hour}) * \text{household number} * 8760$$
$$+ (\Pr(\textit{load} \le 8WM) * \Pr(\textit{point b is unavailable underloaded}))$$
$$* \text{average load per household per hour} + \Pr(\textit{load} \ge 8WM)$$
$$* \Pr(\textit{point b is unavailable overlloaded})$$
$$* \text{average load per household per hour}) * \text{household number} * 8760 +$$

Page: 4

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: Very nice!

Yuan Li - 01228509

(Pr($load \leq 8WM$) * Pr($point\ c\ is\ unavailable\ underloaded$))
* average load per household per hour + Pr($load \geq 8WM$)
* Pr($point\ c\ is\ unavailable\ overloaded$)
* average load per household per hour) * household number * 8760

$$EENS = \left\{\frac{20}{24}\left(1-\left(1-\left(1-\frac{3650}{3651}\right)*\left(1-\frac{3650}{3651}\right)\right)*\frac{1825}{1826}\right)*\frac{8.5}{20}+\right.$$
$$\frac{4}{24}*\left(1-\left(\frac{3650}{3651}*\frac{3650}{3651}*\frac{1825}{1826}+\left(\frac{3650}{3651}*\frac{1}{3651}+\frac{3650}{3651}*\frac{1}{3651}\right)*\frac{1825}{1826}*\left(1-\frac{3650}{3653}\right.\right.\right.$$
$$\left.\left.\left.*\frac{1460}{1461}\right)\right)*\frac{3.25}{4}\right\}*5000*8760$$
$$+\left\{\frac{20}{24}\left(1-\left(1-\left(1-\frac{3650}{3651}\right)*\left(1-\frac{3650}{3651}\right)\right)*\frac{3650}{3653}\right)*\frac{8.5}{20}+\right.$$
$$\frac{4}{24}*\left(1-\left(\frac{3650}{3651}*\frac{3650}{3651}*\frac{3650}{3653}+\left(\frac{3650}{3651}*\frac{1}{3651}+\frac{3650}{3651}*\frac{1}{3651}\right)*\frac{3650}{3653}*\left(1-\frac{1825}{1826}\right.\right.\right.$$
$$\left.\left.\left.*\frac{1460}{1461}\right)\right)*\frac{3.25}{4}\right\}*4000*8760$$
$$+\left\{\frac{20}{24}\left(1-\left(1-\left(1-\frac{3650}{3651}\right)*\left(1-\frac{3650}{3651}\right)\right)*\frac{3650}{3653}*\frac{1460}{1461}\right)*\frac{8.5}{20}+\right.$$
$$\frac{4}{24}*\left(1-\left(\frac{3650}{3651}*\frac{3650}{3651}*\frac{3650}{3653}*\frac{1460}{1461}+\left(\frac{3650}{3651}*\frac{1}{3651}+\frac{3650}{3651}*\frac{1}{3651}\right)*\frac{3650}{3653}*\frac{1460}{1461}\right.\right.$$
$$\left.\left.\left.*\left(1-\frac{1825}{1826}\right)\right)*\frac{3.25}{4}\right\}*2000*8760 = 46.04\text{MWh}$$

5.

Typical numbers for UK distribution network companies lie in the range 25-100 (CI) and 35-70 (CML). Therefore, CML index for this network is not acceptable. The high CML index is because of the relatively low availability of feeders.

ok

Part B

6.

The state includes the six states of its components: up/down for T1, T2, F1, F2, F3 and different load conditions. A sampling routine has been implemented in mc_sample.m.

ok

7.

The program has been implemented in mc_impact.m.

8.

The program has been implemented in mc.m.

ok

Page: 5

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: Is this the average load for peak hours only?

Yuan Li - 01228509

9.

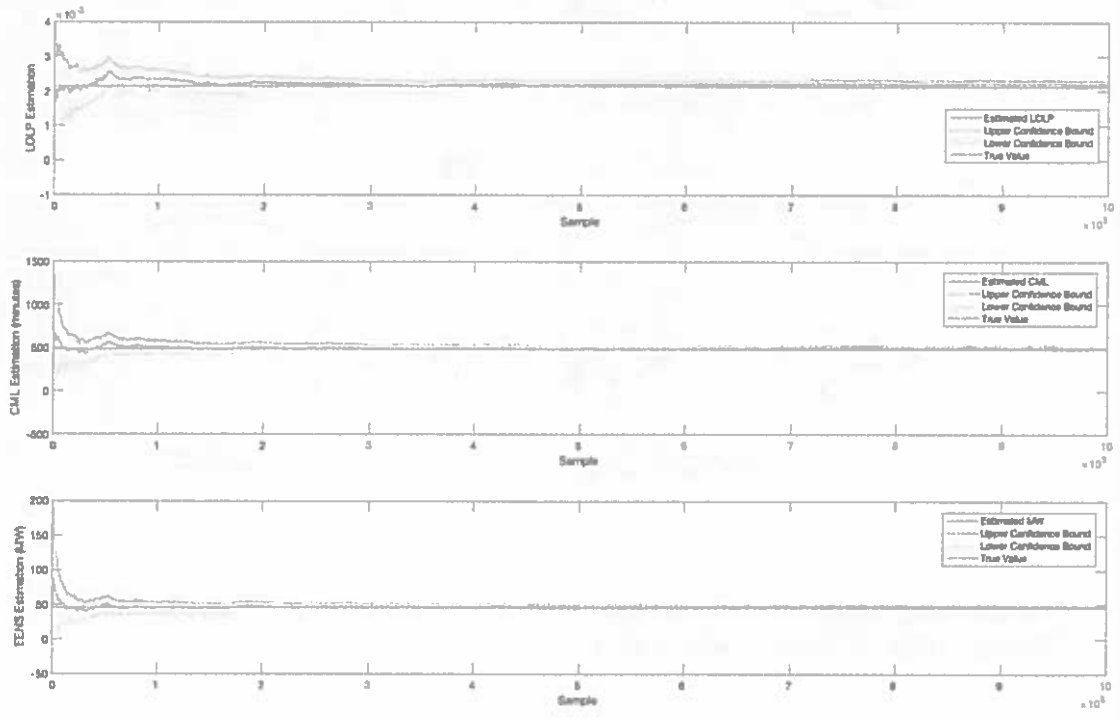
```
lolp = 0.00216 +- 0.00003
lolp = [0.00209, 0.00222] (95% confidence)
lolp = 0.00216 +- 1.5 %
CML = 481.56905 +- 8.08148 min
CML = [465.72935, 497.40876] min (95% confidence)
CML = 481.56905 +- 1.7 % min
EENS = 46.20939 +- 0.88110 MWh
EENS = [44.48244, 47.93635] MWh (95% confidence)
EENS = 46.20939 +- 1.9 % MWh
```

ok

LOLP	0.00216 ± 0.00003
CML	$481.56905 \pm 8.08148\ minutes$
EENS	$46.20939 \pm 0.88110\ MWh$

The simulation with 2,000,000 samples gave the results shown above. The estimated values and the 95% confidence bands are shown in the graph below. The graph implies that the analytical values of LOLP (0.00214), CML (481.64 minutes) and EENS (46.04 MW) are within the expected range of simulated values, and close to the true values.

ok



- Comment

The snapshot approach is easy to program and more clearly compared with analytical solution. However, the results illustrate that achievement of the relatively accuracy estimated values requires large sample size. Moreover, not every metric can be easily measured, such as CI values which is dependent on continuous time. The further comparison between time sequential monte carlo is discussed in question 14.

ok

Part C

10.

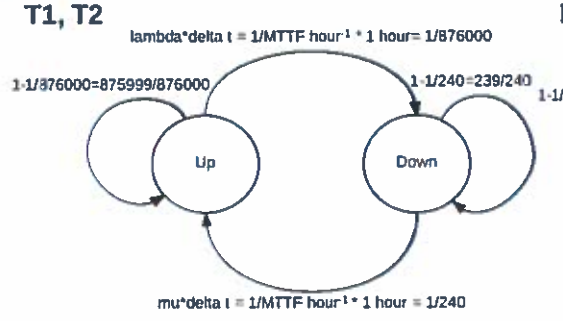
ok

The program has been implemented in mc_time_initial_state.m.

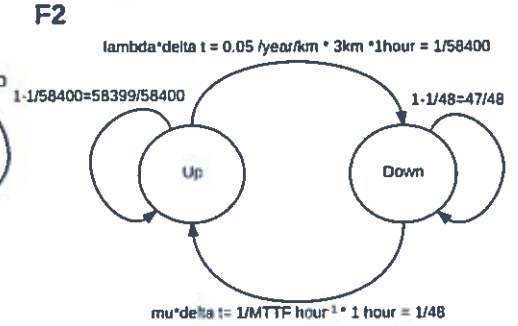
11.

The Markov chains for transformers and feeders are shown below.

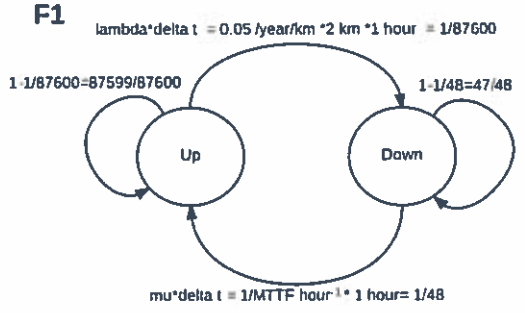
T1, T2



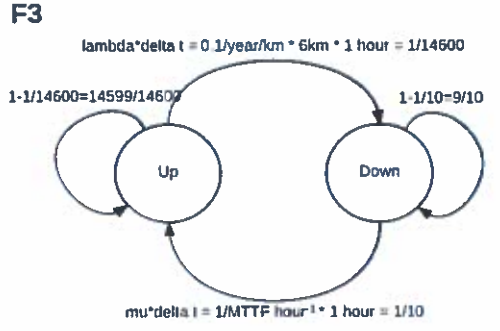
F2



F1



F3



ok

12.

The program has been implemented in mc_time_next_state.m. Note that load level change rate is 1 because $\Delta t = 1\ hour$.

Page: 7

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: technically it's not a rate now: it advances every hour and is not modelled as a stochastic process.

13.

The program has been implemented in mc_time. mc_time_disconnect is used to compute the fractions of customers in three points are disconnected in every instant. It is worth mention that re-interruptions to supply of customers previously interrupted, irrespective of the cause
are excluded by the following codes.

```
%excluding re- interruptions to the supply of customers previously interrupted during the same
incident
ci(year) = ci(year) + (max(0, customer_a - customer_a_old)*5000 + max(0, customer_b -
customer_b_old)*4000 +max(0, customer_c - customer_c_old)*2000)/11000 * 100;
```

Where 'customer_i_old' is the previous fraction of customers not supplied in 'i' point.

ok

14.

```
LOLP = 0.00215 +- 0.00004
LOLP = [0.00207, 0.00223] (95% confidence)
LOLP = 0.00215 +- 1.9 %
CML = 487.13236 +- 11.72976 min
CML = [464.14203, 510.12270] min (95% confidence)
CML = 487.13236 +- 2.4 % min
CI = 48.87273 +- 2.37885 (interruptions / 100 customers)
CI = [44.21018, 53.53527] (interruptions / 100 customers) (95% confidence)
CI = 48.87273 +- 4.9 % (interruptions / 100 customers)
EENS = 47.30169 +- 1.24624 MWh
EENS = [44.85906, 49.74431] MWh(95% confidence)
EENS = 47.30169 +- 2.6 % MWh
```

LOLP	0.00215 ± 0.00004
CML	$487.13236 \pm 11.72976\ minutes$
EENS	$47.30169 \pm 1.24624\ MWh$
CI	$48.87273 \pm 2.37885\ interruptions/100\ customers$

The simulation with independent 10000 years gave the results shown above. The analytical values of LOLP (0.00214), CML (481.64 minutes) and EENS (46.04 MW) are within the expected range of simulated values, and close to the true values. The value of EENS also close to that obtained from state sampling monte carlo ($47.30169 \pm 1.24624 MW$).

- Comment and Comparison

	Running Time (s)
State Sampling Monte Carlo (1000*8760)	190.0s
Time Sequential Monte Carlo (1000 Years)	8.3s
Time Sequential Monte Carlo (50000 Years)	381.3s

8760000 samples are equivalent to 1000-year simulation. Although the running time for Time Sequential Monte Carlo method is much smaller than State Sampling Monte

Page: 8

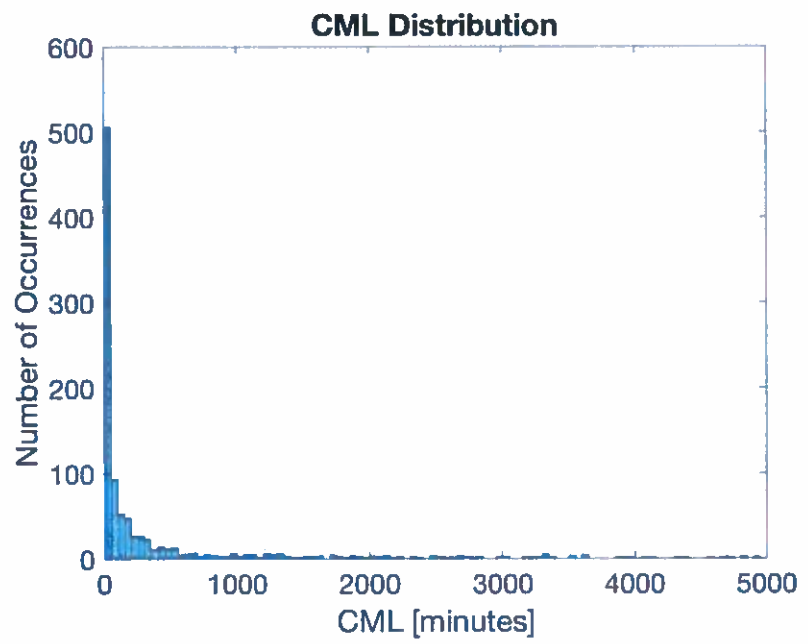
Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: nice

Number: 2 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: arguably, the analytical values ARE the true values

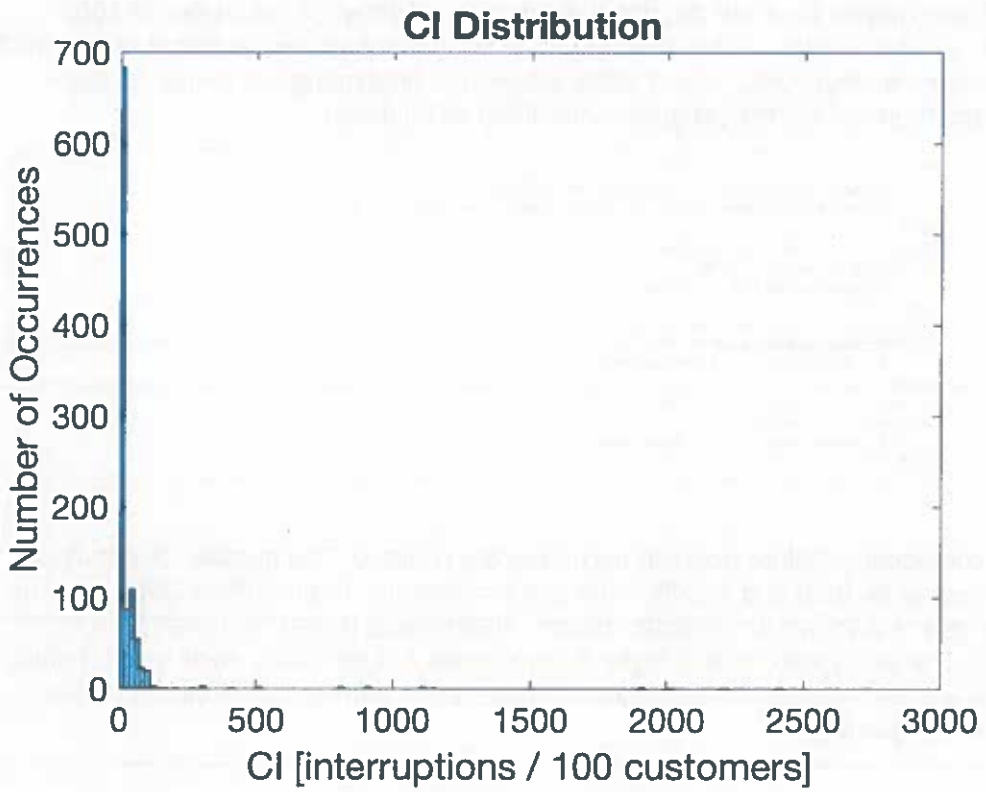
Carlo method, the standard error of the former is 8 times larger than the latter. To achieve the same standard error, the time sequential monte carlo requires 50000-year simulation. The required time for 50000-year simulation is much larger than that of state sampling monte carlo which can achieve the same degree of accuracy. Therefore, the running speed of state sampling monte carlo method is faster than time sequential monte carlo method. However, compared with analytical calculation and state sampling monte carlo, users can easily access to every single time when applying Time sequential monte carlo method, such as calculating CI values which is dependent on the two continuous states. Therefore, time sequential monte carlo is easy to analyze any system property because it is based on continuous time simulation. Moreover, this method can be used to analyze the behaviors of separate years. Finally, users can choose the initial state of the system, which means certain behavior of the system can be analyzed easily.

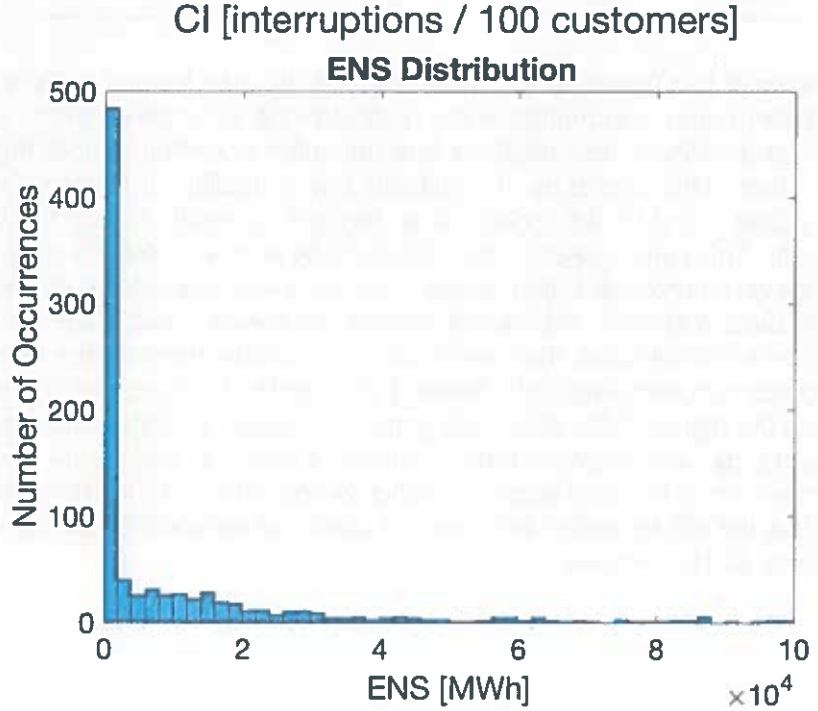
ok

15.



Yuan Li - 01228509





The three graphs illustrate the distribution of three different values out of 1000 years. As can be seen in the three graphs above, there are few extreme years which has extremely high CML, CI and EENS values. For illustrating the causes of these extreme high values, the program is modified as following:

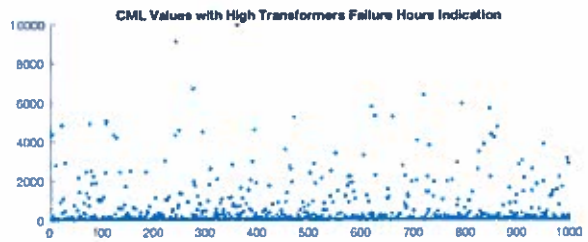
```
%count the hours of one of transformers is down
        if current_state(1)==0||current_state(2)==0
            transformer_down(year) = transformer_down(year) + 1;
        end
        %count the hours of Feeder 1 is down
        if current_state(3)==0
            F1_down(year) = F1_down(year) + 1;
        end
        %count the hours of Feeder 2 is down
        if current_state(4)==0
            F2_down(year) = F2_down(year) + 1;
        end
        %count the hours of Feeder 3 is down
        if current_state(5)==0
            F3_down(year) = F3_down(year) + 1;
        End
```

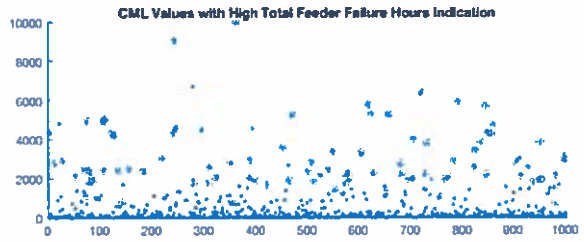
The components failure hours in each year are counted. The number of simulated years is set to 1000 and 10000 in the two simulations. The modified CML values in each year are shown in the figure below, where x-axis is year and y-axis is CML value. The years with several highest transformer failure hours, each feeder failure hours and total feeder failure hours are indicated in different color circles in five figures respectively.

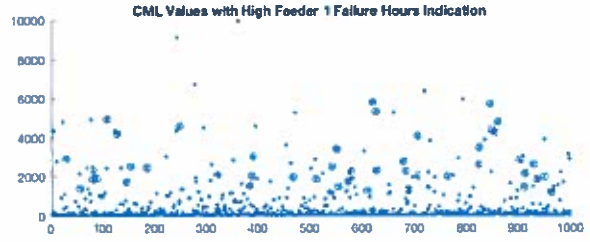
As can be shown in two figures below, both the long duration time of transformer failure and feeder failure contributes to the relatively high CML values of the year. However, the probability of high feeder failure hours is higher than that of high transformer failure. This caused by the relatively low availability of feeders. On the other hand, feeder 1 and feeder 2 rather than feeder 3 generally lead to the high CML of the year. This can be seen by the distributions of three different color circles. The red circles which represent high feeder 3 failure hours concentrate on the bottom, while black and blue circles which denote the feeder 1 and feeder 2 concentrate on the middle part. The overcome is reasonable because the feeder 3 supply less customers compared with feeder 1 and feeder 2/. Moreover, the extreme years that has the highest CML values are generally caused by high transformer failure hours in those year because those years have relatively low feeder failure hours (no green circles in those years). In other words, one of transformers breaks down will affect the whole system because the whole network will break out when overloaded (e.g. at 18, 19 hours).

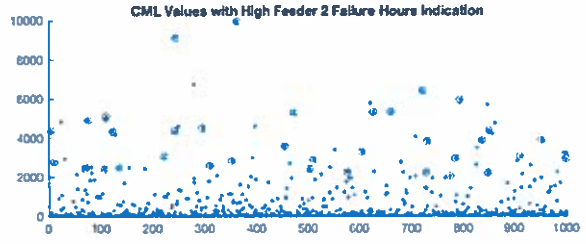
Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: How are these years selected?

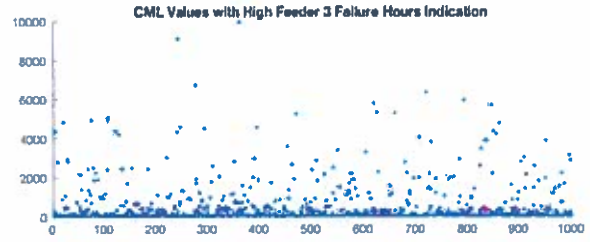
Yuan Li - 01228509

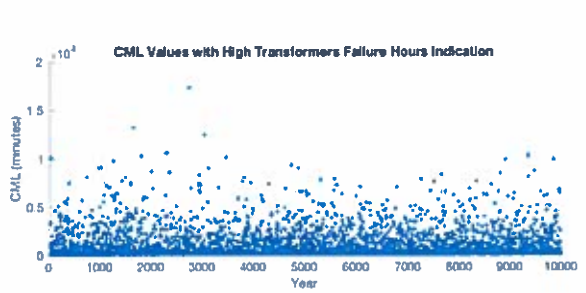


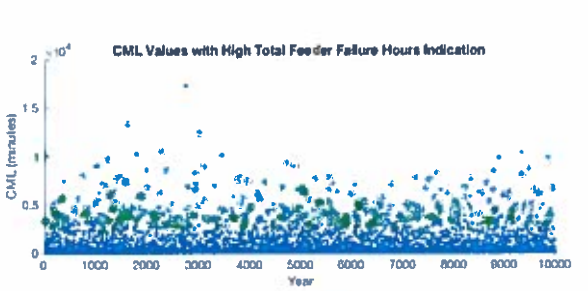


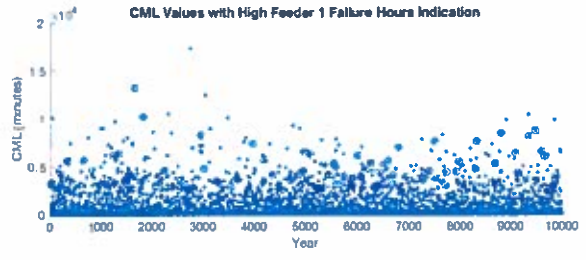


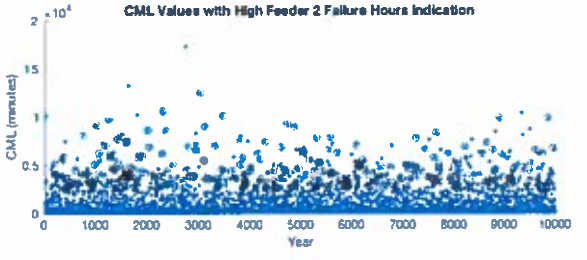


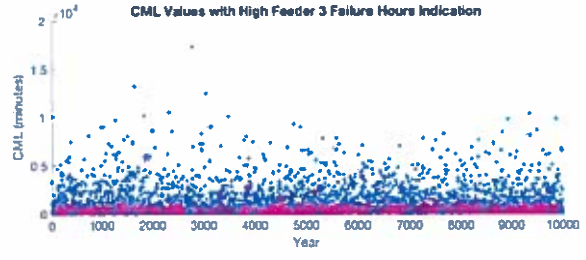












In conclusion, extremely high CML (e.g. 100000min) and CI values[1] are generally caused by long duration failure hours of feeders. This can be caused by the relatively high failure rates of three feeders. However, long duration of failure hours of feeder 1 and feeder 2 will cause relatively high (e.g. 4000-10000min) CML values because the feeder 1 and 2 feeds more customers than feeder 3.

DNO can perform

1. Build more transformers or use more advanced transformers with higher thermal rating.
2. Build more than 1 lines between two locations (N-1 rules).
3. Build normally open point and lines between point a and point b to resupply load points in case of upstream faults (F1 or F2), instead of building lines between point a and point c because the distance between a and b is relatively shorter.
4. Demand response. (e.g. reduce the loads when overloaded).

To perform cost benefit analysis, the following information is required.

1. Interruption cost per event (the money paid to customer during the interruption).
2. The annually cost of extra one transformer or advance transformer.
3. Open point construction and maintenance fee.
4. The total fee of smart meter installed in every household which is used for load prediction during demand response.

To perform the cost benefit analysis, firstly the total interruption cost based on current CML and CI values should be calculated. Then, the cost of extra transformer, demand response and open point construction between point a and point b should be compared. Finally, the future CML and CI values should be predicted for calculating the future interruption cost. The benefit can be defined as:

$$Benefit = (Current\ Interruption\ Cost - future\ Interruption\ Cost) - Cost\ of\ Option$$

Among all the options, building an open point between point a and point b can be consider as the most beneficial one. The CML and CI can be significantly reduced based on the previous discussion on extreme years in this question. Moreover, the construction fee is relatively low compared with other options.

ok

Page: 13

Number: 1 Author: Simon Tindemans Date: Indeterminate
Simon Tindemans: Actually, if you would do the same analysis for CI, you would find that high-CI years are nearly all the result of transformers.

Appendix

mc_sample.m

```
function [ sampled_state ] = mc_sample()
% This function samples a random state of the components and load according
% to their state probabilities.
% Note that I use the fact that the test rand < N returns a value 0 or
% 1, which can be stored directly as the component status (true = up;
% false = down).
    % randomly select components availabilities according to their availabilities
    % using availability = MTTF/MTBF
    F1 = rand < (10*365)/(10*365+2);
    F2 = rand < (20/3*365)/(20/3*365+2);
    F3 = rand < (5/3*8760)/(5/3*8760+10);
    T1 = rand < (100*365)/(100*365+10);
    T2 = rand < (100*365)/(100*365+10);
    load_profile = [0.30 0.25 0.23 0.22 0.21 0.25 0.38 0.52 0.53 0.48 0.45 0.46 0.49 0.44
0.43 0.48 0.65 0.83 0.86 0.81 0.75 0.70 0.60 0.43]*11000;
    %with unit of KW
    hour=randi(24);                          %select random hour
    load = load_profile(hour);
    % return the sampled state as an array with six elements
    sampled_state = [T1 T2 F1 F2 F3 load];

end
```

ok

mc_impact.m

```
function [lol, lol_a, lol_b, lol_c, ens] = mc_impact( state )
    %an overload leads to disconnection of all customers for transformer will trip
    %when overloaded
    ens = 0;                          %initial ens and lol to 0
    lol = 0;                            %lol means one of load points is not supplied

    % first test whether the load can be supplied by a single transformer.
    if state(6) <= 8000                    %load can be supplied by a single
transformer  (with unit of kw)
        % if the feeder 1 is down customers in point a are lost or both
        % transformer are down
        if (not(state(1)) && not(state(2))) || not(state(3))
            lol_a = 1;
            lol = 1;
            ens = state(6)/11000*5000 + ens;         %customers in point a are all
disconnected
        else
            lol_a = 0;                                %otherwise customers in point a are
supplied
        end

        % if the feeder 2 is down customers in point b are lost or both
        % transformer are down
        if (not(state(1)) && not(state(2))) || not(state(4))
            lol_b = 1;
            lol = 1;
            ens = state(6)/11000*4000 + ens;     %customers in point b are all disconnected
        else
            lol_b = 0;                               %otherwise customers in point b are
supplied
        end

        % if the feeder 1 or feeder 2 is down customers in point a are lost or both
        % transformer are down
        if (not(state(1)) && not(state(2))) || not(state(4)) || not(state(5))
            lol_c = 1;
            lol = 1;
            ens = state(6)/11000*2000 + ens;      %customers in point a are all disconnected
        else
            lol_c = 0;                                %otherwise customers in point a are
supplied
        end
    else                                             %load cannot be supplied by a single
transformer
```

```
        % if the feeder 1 and both transfomers are up customers in point a
        % are supplied
        if state(1) && state(2) && state(3)
            lol_a = 0;

        % if the feeder 1 is up, only one of transfomers is up and one of other two feeders
(F2 and F3) is
        %unavailable customers in point a are supplied
        elseif (state(1)&& not(state(2)) || not(state(1)) && state(2)) && state(3) &&
(not(state(4)) || not(state(5)))
            lol_a = 0;

        %otherwise customers in point a are not supplied
        else
            lol_a = 1;
            lol = 1;
            ens = state(6)/11000*5000 + ens;
        end

        % if the feeder 2 and both transfomers are up customers in point b
        % are supplied
        if state(1) && state(2) && state(4)
            lol_b = 0;

        % if the feeder 2 is up, only one of transfomers is up and one of other two feeders
(F1 and F3) is
        %unavailable customers in point b are supplied
        elseif (state(1)&& not(state(2)) || not(state(1)) && state(2)) && state(4) &&
(not(state(3)) || not(state(5)))
            lol_b = 0;

        %otherwise customers in point b are not supplied
        else
            lol_b = 1;
            lol = 1;
            ens = state(6)/11000*4000 + ens;
        end

        % if the feeder 2, feeder 3 and both transfomers are up customers
        % in point c are supplied
        if state(1) && state(2) && state(4) && state(5)
            lol_c = 0;

        % if the feeder 2 and feeder 3 are up, only one of transfomers is up and feeder 1
is
        %unavailable customers in point c are supplied
        elseif (state(1)&& not(state(2)) || not(state(1)) && state(2)) && state(4) &&
state(5) && not(state(3))
            lol_c = 0;

        %otherwise customers in point c are not supplied
        else
            lol_c = 1;
            lol = 1;
            ens = state(6)/11000*2000 + ens;
        end

    end
end
```

ok

mc.m

```
tic;
T = 8760000;           %sample sizes
% initialise empty arrays
lol_a = zeros(1, T);                          %point a, b and c not supply
lol_b = zeros(1, T);
lol_c = zeros(1, T);
lol = zeros(1, T);                              %lolp
ens = zeros(1, T);
CML = zeros(1, T);
for i=1:T
    % generate samples and compute five measurements each time
    state = mc_sample();
```

```
    [lol(i), lol_a(i), lol_b(i), lol_c(i), ens(i)] = mc_impact(state);
    CML(i) = 8760 * 60 * (5000 * lol_a(i) + 4000 * lol_b(i) +2000 * lol_c(i)) / 11000;
end
toc;
% compute lolp of every point and se
lolp_a_mean = mean(lol_a);
lolp_b_mean = mean(lol_b);
lolp_c_mean = mean(lol_c);
lolp_mean = mean(lol);
lolp_a_se = std(lol_a)/sqrt(T);
lolp_b_se = std(lol_b)/sqrt(T);
lolp_c_se = std(lol_c)/sqrt(T);
lolp_se = std(lol)/sqrt(T);

% compute excepted CML and se
CML_mean = mean(CML);
CML_se = std(CML)/sqrt(T);

% compute EENS values and se
eens_mean = 8760 * mean(ens)/10^3;
eens_se = 8760 * (std(ens)/sqrt(T))/10^3;

% print the results to screen (with 95% confidence intervals)
fprintf('lolp = %.5f +- %.5f\n',lolp_mean, lolp_se);
fprintf('lolp = [%.5f, %.5f] (95%% confidence)\n',lolp_mean - 1.96*lolp_se, lolp_mean +
1.96*lolp_se);
fprintf('lolp = %.5f +- %.1f %%\n',lolp_mean, 100*lolp_se/lolp_mean);

fprintf('CML = %.5f +- %.5f min\n',CML_mean, CML_se);
fprintf('CML = [%.5f, %.5f] min (95%% confidence)\n',CML_mean - 1.96*CML_se, CML_mean +
1.96*CML_se);
fprintf('CML = %.5f +- %.1f %% min\n',CML_mean, 100*CML_se/CML_mean);

fprintf('EENS = %.5f +- %.5f MWh\n',eens_mean, eens_se);
fprintf('EENS = [%.5f, %.5f] MWh(95%% confidence)\n',eens_mean - 1.96*eens_se, eens_mean +
1.96*eens_se);
fprintf('EENS = %.5f +- %.1f %% MWh\n',eens_mean, 100*eens_se/eens_mean);

%partial mean and standard error of lolp, cml and eens are stored in six
%arrays for comparing with their values
% Use only one in 1000 samples to speed things up
partial_means_lolp = zeros(1, T/1000);
partial_se_lolp = zeros(1, T/1000);
for i=1:T/1000
    % compute partial means and partial standard errors
    partial_means_lolp(i) = mean(lol(1:1000*i));
    partial_se_lolp(i) = std(lol(1:1000*i))/sqrt(1000*i);
end
partial_means_CML = zeros(1, T/1000);
partial_se_CML = zeros(1, T/1000);
for i=1:T/1000
    % compute partial means and partial standard errors
    partial_means_CML(i) = mean(CML(1:1000*i));
    partial_se_CML(i) = std(CML(1:1000*i))/sqrt(1000*i);
end
partial_means_eens = zeros(1, T/1000);
partial_se_eens = zeros(1, T/1000);
for i=1:T/1000
    % compute partial means and partial standard errors
    partial_means_eens(i) = mean(8760*ens(1:1000*i)/10^3);
    partial_se_eens(i) = std(8760*ens(1:1000*i)/10^3)/sqrt(1000*i);
end

% plot the true value, the MC estimate and the 95% confidence bands
% for visualising the convergence
figure;
subplot(3,1,1);
plot(1:1000:T, partial_means_lolp);                          %step by 100 to speed things up
hold on;
plot(1:1000:T, partial_means_lolp + 1.96 * partial_se_lolp);
plot(1:1000:T, partial_means_lolp - 1.96 * partial_se_lolp);
plot([1 T] , [0.00214 0.00214]);
ylabel('LOLP Estimation');
xlabel('Sample');
legend('Estimated LOLP','Upper Confidence Bound','Lower Confidence Bound','True Value');

subplot(3,1,2);
plot(1:1000:T, partial_means_CML);                           %step by 100 to speed things up
```

```
hold on;
plot(1:1000:T, partial_means_CML + 1.96 * partial_se_CML);
plot(1:1000:T, partial_means_CML - 1.96 * partial_se_CML);
plot([1 T] , [481.64 481.64]);
ylabel('CML Estimation (minutes)');
xlabel('Sample');
legend('Estimated CML','Upper Confidence Bound','Lower Confidence Bound','True Value');

subplot(3,1,3);
plot(1:1000:T, partial_means_eens);                          %step by 100 to speed things up
hold on;
plot(1:1000:T, partial_means_eens + 1.96 * partial_se_eens);
plot(1:1000:T, partial_means_eens - 1.96 * partial_se_eens);
plot([1 T] , [46.04 46.04]);
ylabel('EENS Estimation (MW)');
xlabel('Sample');
legend('Estimated MW','Upper Confidence Bound','Lower Confidence Bound','True Value');
```

ok

mc_time_initial_state.m

```
function [ initial_state ] = mc_time_initial_state()
%This function generates initial sample of the components, load and hour according
%to their state probabilities.
%i.e. rand < 0.98 returns a value 0 or 1, which can be stored directly as
%the component status (true = up; false = down).   availability = MTTF/MTBF
    F1 = rand < (10*365)/(10*365+2);
    F2 = rand < (20/3*365)/(20/3*365+2);
    F3 = rand < (5/3*8760)/(5/3*8760+10);
    T1 = rand < (100*365)/(100*365+10);
    T2 = rand < (100*365)/(100*365+10);
    load_state = 1;                                  %initial hour with 1
    initial_state = [T1 T2 F1 F2 F3 load_state];     % return the initial state as an
array with seven elements

end
```

ok

mc_time_next_state.m

```
function [ next_state ] = mc_time_next_state( current_state )
%Takes the current state and returns the state after 1 hour

    % define the failure and repair rates (in units of 1/hour)
    % component order is: T1 T2 F1 F2 F3
    rates_fail = [1/876000 1/876000 1/87600 1/58400 1/14600];
    rates_repair = [1/240 1/240 1/48 1/48 1/10];
    dt = 1;                                          %one time step dt is 1 hour
    rates_load = 1;

    % start with the old state, and only update components that change
    next_state = current_state;

    % Transformer 1
    if current_state(1)==1                %transformer 1 is up
        if rand() < rates_fail(1)*dt
            next_state(1) = 0;
        end
    else                                  %transformer 1 is down
        if rand() < rates_repair(1)*dt
            next_state(1) = 1;
        end
    end

    % Transformer 2
    if current_state(2)==1                %transformer 2 is up
        if rand() < rates_fail(2)*dt
            next_state(2) = 0;
        end
    else                                  %transformer 2 is down
        if rand() < rates_repair(2)*dt
            next_state(2) = 1;
        end
    end

    % Feeder 1
    if current_state(3)==1                % feeder 1 is available
```

```
        if rand < rates_fail(3)*dt
            next_state(3) = 0;
        end
    else                            % feeder 1 is unavailable
        if rand < rates_repair(3)*dt
            next_state(3) = 1;
        end
    end

    % Feeder 2
    if current_state(4)==1                  % feeder 2 is available
        if rand < rates_fail(4)*dt
            next_state(4) = 0;
        end
    else                            % feeder 2 is unavailable
        if rand < rates_repair(4)*dt
            next_state(4) = 1;
        end
    end

    % Feeder 3
    if current_state(5)==1                  % feeder 3 is available
        if rand() < rates_fail(5)*dt
            next_state(5) = 0;
        end
    else                            % feeder 3 is unavailable
        if rand() < rates_repair(5)*dt
            next_state(5) = 1;
        end
    end

    % load level
    if current_state(6) < 24
        if rand() < rates_load*dt
            next_state(6) = current_state(6) + 1;
        end
    else                                        %become a cycle, hour 24 return to hour 1
        if rand() < rates_load*dt
            next_state(6) = 1;
        end
    end
end
```

ok

mc_time_disconnect.m

```
function [ customers_dis_a, customers_dis_b, customers_dis_c, lol ] =
mc_time_disconnect( state )
    %Computes the fraction of customers in three points that is disconnected in 'state'.
    %Note that an overload leads to disconnection of all customers for transformer will
trip
    %when overloaded

    load_profile = [0.30 0.25 0.23 0.22 0.21 0.25 0.38 0.52 0.53 0.48 0.45 0.46 0.49 0.44
0.43 0.48 0.65 0.83 0.86 0.81 0.75 0.70 0.60 0.43];
    load = load_profile(state(6))*11000;                %determine the maximum total load
level of all household
    customers_dis_a = 0;                                %initial with all customers are
supplied
    customers_dis_b = 0;
    customers_dis_c = 0;
    lol = 0;                                           %lol means one of load point is not
supplied

    % first test whether the load can be supplied by a single transformer.
    if load <= 8000    %kw
        % if the feeder 1 is down customers in point a are lost or both
        % transformer are down
        if (not(state(1)) && not(state(2))) || not(state(3))
            customers_dis_a = 1;
            lol = 1;
        end

        % if the feeder 2 is down customers in point b are lost or both
        % transformer are down
        if (not(state(1)) && not(state(2))) || not(state(4))
```

```
            customers_dis_b = 1;
            lol = 1;
        end

        % if the feeder 1 or feeder 2 is down customers in point a are lost or both
        % transformer are down
        if (not(state(1)) && not(state(2))) || not(state(4)) || not(state(5))
            customers_dis_c = 1;
            lol = 1;
        end

        %load cannot be supplied by a single transformer
    else
        % if the feeder 1 and both transfomers are up customers in point a
        % are supplied
        if state(1) && state(2) && state(3)

        % if the feeder 1 is up, only one of transfomers is up and one of other two feeders
(F2 and F3) is
        %unavailable customers in point a are supplied
        elseif (state(1)&& not(state(2)) || not(state(1)) && state(2)) && state(3) &&
(not(state(4)) || not(state(5)))

            %otherwise customers in point a are not supplied
        else
            customers_dis_a = 1;
            lol = 1;
        end

        % if the feeder 2 and both transfomers are up customers in point b
        % are supplied
        if state(1) && state(2) && state(4)

        % if the feeder 2 is up, only one of transfomers is up and one of other two feeders
(F1 and F3) is
        %unavailable customers in point b are supplied
        elseif (state(1)&& not(state(2)) || not(state(1)) && state(2)) && state(4) &&
(not(state(3)) || not(state(5)))

        %otherwise customers in point b are not supplied
        else
            customers_dis_b = 1;
            lol = 1;
        end

        % if the feeder 2, feeder 3 and both transfomers are up customers
        % in point c are supplied
        if state(1) && state(2) && state(4) && state(5)

        % if the feeder 2 and feeder 3 are up, only one of transfomers is up and feeder 1
is
        %unavailable customers in point c are supplied
        elseif (state(1)&& not(state(2)) || not(state(1)) && state(2)) && state(4) &&
state(5) && not(state(3))

        %otherwise customers in point c are not supplied
        else
            customers_dis_c = 1;
            lol = 1;
        end

    end

end
```

ok

mc_time.m

```
tic;
T = 50000;                % number of independent years to simulate

% initialise cml and ci arrays to zero
cml = zeros(1,T);
ci = zeros(1,T);
lolp = zeros(1,T);
eens = zeros(1,T);
```

```
transformer_down = zeros(1,T);          %count the hours of one of transformers is down in each
year
F2_down = zeros(1,T);
F1_down = zeros(1,T);                   %count the hours of F1, F2 and F3 are down in each year
F3_down = zeros(1,T);

load_profile = [0.30 0.25 0.23 0.22 0.21 0.25 0.38 0.52 0.53 0.48 0.45 0.46 0.49 0.44 0.43 0.48 0.65
0.83 0.86 0.81 0.75 0.70 0.60 0.43];

for year = 1:T
    %initial state with hour 1
    current_state = mc_time_initial_state();
    [customer_a, customer_b, customer_c] = mc_time_disconnect(current_state);


    for time = 1:8760                   %one year have 24*365 hours
        customer_a_old = customer_a;            % store the previously disconnected customer
fraction. This is used
        customer_b_old = customer_b;            % for the CI computation.
        customer_c_old = customer_c;

        % compute the *current* fraction of disconnected customers, under
        % both load shedding assumptions.
        [customer_a, customer_b, customer_c] = mc_time_disconnect(current_state);

        % compute the contribution of the time slice(1 hour) to the CML and CI
        % indices. Note that CML is computed in minutes, and CI is the
        % number of disconnections for 100 average customers
        cml(year) = cml(year) + 60*(customer_a*5000 + customer_b*4000 +customer_c*2000)/11000;

        %excluding re- interruptions to the supply of customers previously interrupted during the
same incident
        ci(year) = ci(year) + (max(0, customer_a - customer_a_old)*5000 + max(0, customer_b -
customer_b_old)*4000 +max(0, customer_c - customer_c_old)*2000)/11000 * 100;

        %loss of load probability of every hour
        lolp(year) = lolp(year) + max(max(customer_a, customer_b), customer_c);

        eens(year) = eens(year) + load_profile(current_state(6))*(customer_a*5000 + customer_b*4000
+customer_c*2000);

        %count the hours of one of transformers is down
        if current_state(1)==0||current_state(2)==0
            transformer_down(year) = transformer_down(year) + 1;
        end
        %count the hours of Feeder 1 is down
        if current_state(3)==0
            F1_down(year) = F1_down(year) + 1;
        end
        %count the hours of Feeder 2 is down
        if current_state(4)==0
            F2_down(year) = F2_down(year) + 1;
        end
        %count the hours of Feeder 3 is down
        if current_state(5)==0
            F3_down(year) = F3_down(year) + 1;
        end

        %advance the state for one hour
        current_state = mc_time_next_state(current_state);
    end
end
toc;

% compute excepted CML and se
cml_mean = mean(cml);
cml_se = std(cml)/sqrt(T);

% compute CI values and se
ci_mean = mean(ci);
ci_se = std(ci)/sqrt(T);

% compute lolp and se
lolp_mean = mean(lolp/8760);
lolp_se = std(lolp/8760)/sqrt(T);

% compute EENS values and se
eens_mean = mean(eens/1e3);
eens_se = std(eens/1e3)/sqrt(T);

% print the results to screen (with 95% confidence intervals)
fprintf('LOLP = %.5f +- %.5f\n',lolp_mean, lolp_se);
fprintf('LOLP = [%.5f, %.5f] (95%% confidence)\n',lolp_mean - 1.96*lolp_se, lolp_mean +
1.96*lolp_se);
```

```
fprintf('LOLP = %.5f +- %.1f %%\n',lolp_mean, 100*lolp_se/lolp_mean);

fprintf('CML = %.5f +- %.5f min\n',cml_mean, cml_se);
fprintf('CML = [%.5f, %.5f] min (95%% confidence)\n',cml_mean - 1.96*cml_se, cml_mean +
1.96*cml_se);
fprintf('CML = %.5f +- %.1f %% min\n',cml_mean, 100*cml_se/cml_mean);

fprintf('CI = %.5f +- %.5f (interruptions / 100 customers)\n',ci_mean, ci_se);
fprintf('CI = [%.5f, %.5f] (interruptions / 100 customers) (95%% confidence)\n',ci_mean -
1.96*ci_se, ci_mean + 1.96*ci_se);
fprintf('CI = %.5f +- %.1f %% (interruptions / 100 customers)\n',ci_mean, 100*ci_se/ci_mean);

fprintf('EENS = %.5f +- %.5f MWh\n',eens_mean, eens_se);
fprintf('EENS = [%.5f, %.5f] MWh(95%% confidence)\n',eens_mean - 1.96*eens_se, eens_mean +
1.96*eens_se);
fprintf('EENS = %.5f +- %.1f %% MWh\n',eens_mean, 100*eens_se/eens_mean);


%draw histograms of CML and CI
figure;
xbins=0:50:5000;                    %adjust the scale of x
histogram(cml,xbins);
xlabel('CML [minutes]');
ylabel('Number of Occurrences');
title('CML Distribution');
set(gca,'fontsize',18);

figure;
xbins=0:20:3000;
histogram(ci,xbins);
xlabel('CI [interruptions / 100 customers]');
title('CI Distribution');
ylabel('Number of Occurrences');
set(gca,'fontsize',18);

figure;
xbins=0:2000:100000;
histogram(eens,xbins);
xlabel('ENS [MWh]');
title('ENS Distribution');
ylabel('Number of Occurrences');
set(gca,'fontsize',18);

%find the years of high transformers failure hours (transformer failure hours is higher than
%20% of the maximum one) in T years
year_tran_max = find(transformer_down>=max(transformer_down)/5);
%find the years of high feeder 1 failure hours (feeder failure hours is higher than
%20% of the maximum one) in T years
year_F1_max = find(F1_down>=max(F1_down)/5);
year_F2_max = find(F2_down>=max(F2_down)/5);
year_F3_max = find(F3_down>=max(F3_down)/5);
%find the years of high total feeder failure hours (feeder failure hours is higher than
%20% of the maximum one) in T years
year_F_max = find((F1_down+F2_down+F3_down)>=max(F1_down+F2_down+F3_down)/5);
m = size(year_tran_max);
n = size(year_F_max);
x = size(year_F1_max);
y = size(year_F2_max);
z = size(year_F3_max);

%plot CML values with causes indication
figure;
subplot(3,2,1);
ymax = get(gca,'ylim');
hold on;
for i=1:1:m(2)
    plot([year_tran_max(i) year_tran_max(i)],cml(year_tran_max(i)),'ro');
end
title('CML Values with High Transformers Failure Hours Indication');
plot(cml,'.');
xlabel('Year');
ylabel('CML (minutes)');

subplot(3,2,2);
hold on;
plot(cml,'.');
for i=1:1:n(2)
    plot([year_F_max(i) year_F_max(i)],cml(year_F_max(i)),'go');
end
title('CML Values with High Total Feeder Failure Hours Indication');
xlabel('Year');
ylabel('CML (minutes)');
```

ok

Yuan Li - 01228509

```
subplot(3,2,3);
hold on;
plot(cml,'.');
for i=1:1:x(2)
    plot([year_F1_max(i) year_F1_max(i)],cml(year_F1_max(i)),'bo');
end
title('CML Values with High Feeder 1 Failure Hours Indication');
xlabel('Year');
ylabel('CML (minutes)');

subplot(3,2,4);
hold on;
plot(cml,'.');
for i=1:1:y(2)
    plot([year_F2_max(i) year_F2_max(i)],cml(year_F2_max(i)),'ko');
end
title('CML Values with High Feeder 2 Failure Hours Indication');
xlabel('Year');
ylabel('CML (minutes)');

subplot(3,2,5);
hold on;
plot(cml,'.');
for i=1:1:z(2)
    plot([year_F3_max(i) year_F3_max(i)],cml(year_F3_max(i)),'mo');
end
title('CML Values with High Feeder 3 Failure Hours Indication');
xlabel('Year');
ylabel('CML (minutes)');
```

ok